

Load-Balanced Parallel Information Retrieval Using Pthreads, OpenMP, and MPI

Saroosh Javed

*Department of Computer Science
University of Engineering and Technology
Lahore, Pakistan
2023cs67@student.uet.edu.pk*

Waqas Ali

*Department of Computer Science
University of Engineering and Technology
Lahore, Pakistan
waqas.ali2@uet.edu.pk*

Abstract—A sequential information retrieval pipeline that combines tokenization, spell correction, inverted index construction, TF-IDF scoring, keyword search, and co-occurrence analysis turns into a real bottleneck once document collections get large. This paper parallelizes exactly that pipeline using three established models, Pthreads, OpenMP, and MPI, with a hybrid MPI plus OpenMP version added for comparison. Every model splits work at the document level, and a Longest Processing Time greedy scheduler is applied at the MPI level to fix the load imbalance caused by highly uneven document sizes. A lock-free merge strategy partitions hash-table buckets across workers so that no bucket is ever written by more than one thread or process, which removes most of the synchronization overhead in the reduction phase. The experiments ran on a corpus of forty-nine public-domain books, about forty-four megabytes of text in total, checked against a dictionary of over 370,000 English words. The sequential baseline took roughly 2163 seconds. Pthreads reached the highest speedup at approximately 21.50 times using eight threads, ahead of OpenMP at approximately 16.25 times also at eight threads, while MPI reached approximately 11.45 times at four processes. Synchronization overhead, communication cost, and load balancing turned out to be the main factors limiting scalability across the three models, and programming effort and synchronization complexity are compared qualitatively as well.

Index Terms—parallel computing, MPI, OpenMP, Pthreads, information retrieval, inverted index, load balancing, TF-IDF

I. INTRODUCTION

Building an inverted index, scoring terms by their weight, and counting how often words appear near each other are core operations behind almost every information retrieval system [1]. Term-weighting schemes like TF-IDF are not the only way to rank documents, link-based methods such as PageRank take a completely different approach for hyperlinked collections [18], but for a plain text corpus with no link structure, term-weighting is the natural fit and the one this paper builds on. None of these are cheap once a text collection grows

past a trivial size, and a purely sequential implementation eventually turns into the bottleneck that limits how responsive the whole system can be [14]. Parallel and distributed execution is the standard answer to that problem, since spreading the same work across multiple cores or machines lets larger collections finish within a reasonable time [16].

This split between shared-memory and distributed-memory hardware is the same one Flynn used to classify computer architectures [17], and it maps directly onto how the three models manage parallelism. On a single shared-memory machine, two models are typically used to express that parallelism: POSIX threads (Pthreads), where the programmer manages thread creation and synchronization directly [10], and OpenMP, where compiler directives generate the equivalent threaded code with far less manual bookkeeping [9]. When memory is not shared, across separate processes or separate machines, the Message Passing Interface (MPI) is the dominant choice [8]. General treatments of all three models side by side exist [11], but none of the three is strictly better than the others; each trades off implementation effort against synchronization overhead and scalability in a different way [19], [32], which is exactly what this paper sets out to measure directly rather than assume.

This paper parallelizes a complete information retrieval pipeline: tokenization, dictionary-based spell correction via edit distance [4], inverted index construction [14], TF-IDF scoring [2], [33], keyword search, and sliding-window word co-occurrence counting of the kind used to train word embedding models [3]. Two problems specific to this pipeline make it more than a routine parallelization exercise. The first is that spell correction needs a cache shared across all workers, and that cache is mutable, so it has to be synchronized correctly without becoming a bottleneck of its own; prior work on parallel indexing rarely treats this case [24], [25]. The

second is that real document collections are almost never uniform in size, and a handful of unusually large files can dominate the runtime if work is split up naively [21], [22]. The second problem is solved here using a Longest Processing Time (LPT) greedy scheduler [6], [7], described in Section IV.

The rest of the paper follows this structure. Section II surveys related work on comparing parallel paradigms and on parallel information retrieval specifically. Section III walks through the sequential baseline, its pseudocode, and a complexity analysis. Section IV covers the parallel design of all three required models plus the bonus hybrid version. Section V lays out the experimental setup, the dataset, and exactly which input sizes were tested. Section VI reports the strong- and weak-scaling results. Section VII discusses why the numbers came out the way they did. Section VIII closes with the main takeaways and a direction for future work.

II. LITERATURE REVIEW

This section walks through recent research on parallel approaches to information retrieval, touching on inverted index construction, multi-paradigm performance comparisons, parallel spell correction, and load balancing for uneven workloads.

AlHafez and Kurdi [19] compared MPI, OpenMP, and CUDA across a range of HPC application domains, finding that MPI scales well on distributed clusters at the cost of communication overhead, while OpenMP runs efficiently on shared memory until thread counts get high enough for memory contention to bite. What their survey does not do is apply any of the three paradigms to an information retrieval workload specifically, which is precisely the gap this paper fills by running the same three-way comparison on an inverted indexing, spell correction, and co-occurrence pipeline.

Kang et al. [20] took a different but related approach, implementing an all-pairs-shortest-path computation and a large-scale join operation separately in OpenMP, MPI, and MapReduce, then measuring real cluster performance for each. Their study is a useful precedent for direct multi-paradigm comparison, even though their chosen problems, graph traversal and database joins, have little to do with text retrieval. The work here follows the same comparative methodology but applies it to a more communication-heavy, text-centric workload instead.

Guana and Davidson [21] built an inverted index twice, once in Hadoop and once in MPI, measuring execution time, network activity, and disk usage across datasets of varying size and density. Hadoop generally outperformed their MPI version, and adding a pre-merge combiner step helped both, but notably their MPI implementation never accounted for documents of unequal

size. That specific omission is exactly what the LPT scheduler in this paper is designed to fix.

Silveira et al. [22] took the hardware route, proposing a heterogeneous CPU and GPU architecture for inverted index generation that separates compute nodes from a centralizing merge node and reports speedups of up to 60 times over a sequential baseline. That result is hard to argue with, but it depends on GPU hardware and a multi-node cluster that not every project has access to. This paper gets a comparable separation between compute and merge responsibilities using only CPU-based Pthreads, OpenMP, and MPI.

The NASA Advanced Supercomputing Division report [23] is still a standard reference point for comparing OpenMP, MPI, and hybrid paradigms, built around the NAS Parallel Benchmarks. It is foundational but dated; its benchmark suite predates the text-processing workloads that are now common. The three-paradigm comparison method it established is carried forward here, just applied to a text-analytics pipeline instead.

A parallel spell-checking algorithm built on Yahoo N-Grams data [24] split spell checking into three parallel sub-algorithms, one each for error detection, candidate generation, and contextual correction, and proposed (without implementing) a distributed extension. What none of the existing spell-correction literature does is integrate that correction step directly inside a parallel indexing loop behind a shared, synchronized cache, which is exactly what happens here, protected by a double-checked locking pattern that avoids serializing every repeated lookup.

Mei et al. [26] introduced compact data structures for fast lookup of co-occurrence statistics and pairwise word relatedness, aimed mainly at the processing-time and memory cost of naive co-occurrence computation on a single machine. Their work is about single-machine efficiency rather than parallel decomposition. A similar hash-table-based co-occurrence structure is used here, but its construction is parallelized directly across threads and processes using the same lock-free bucket-partitioned merge built for the inverted index.

Zhang and Ding [27] used Apache Spark to scale a co-occurrence-based hot-topic mining algorithm to large social media datasets, reporting substantial gains over a single-node implementation. That result fits a broader shift toward big-data frameworks for this kind of analysis, which sits in contrast to the classical HPC paradigms used here. The contrast is itself worth noting: this project's results show that plain MPI and OpenMP parallelism still holds up well for moderately sized natural-language corpora, without needing a distributed big-data framework at all.

Piereck et al. [28] processed the entire MEDLINE abstract collection in six days on HPC infrastructure,

against an estimated two years for the original sequential tool, a striking illustration of how much speedup is on the table once a text-mining pipeline is properly parallelized. Their domain is biomedical literature mining rather than general information retrieval, and the numbers they report describe end-to-end throughput rather than a controlled comparison across paradigms.

Della Croce and Scatamacchia [7] went back to Graham’s original Longest Processing Time bound [6] and tightened the classical worst-case approximation ratio. Their contribution is purely theoretical, with no application to a specific parallel system. Here, the underlying LPT heuristic is put to direct, practical use in scheduling documents for the MPI and hybrid implementations, with the resulting load-balance improvements actually measured rather than just bounded analytically.

A study comparing a custom MPI and OpenMP MapReduce implementation against Apache Spark on a word-count task [31] found the custom version roughly an order of magnitude faster for that workload. That result backs up the choice made in this paper to stick with classical HPC paradigms rather than a big-data framework, given the moderate scale of the test corpus.

A proposal for a standard interface for user-defined scheduling in OpenMP [29] argued that the built-in static, dynamic, and guided scheduling clauses fall short for workloads with irregular, size-dependent costs, and sketched out an interface for programmers to plug in custom scheduling policies. It stops at the proposal stage and is never evaluated on a real application. The measured gap between Pthreads’ hand-written document-claiming strategy and OpenMP’s built-in dynamic scheduling, reported in Section VII-B, is essentially a concrete, measured instance of the exact scheduling-overhead problem this proposal is trying to solve.

A study of load-balancing strategies for flowshop scheduling on parallel architectures [30] classified static and dynamic load-balancing heuristics and tested them on flowshop scheduling problems, a domain quite far removed from document-level information retrieval. The same static-versus-dynamic distinction is used here: a static, sorted greedy heuristic at the MPI level, where assignment has to be fixed before execution starts, and a dynamic claiming strategy at the Pthreads and OpenMP levels, where shared memory makes runtime work-stealing possible.

The comparative analysis is presented in Table I, which positions the proposed system against the methods discussed above.

Three patterns stand out across this body of work. Direct multi-paradigm comparisons do exist, but they almost always pick graph algorithms, database joins, or generic HPC benchmarks as the test workload rather than information retrieval itself. Parallel spell correction has

TABLE I: Comparison of the proposed parallel information retrieval system with existing state-of-the-art parallel and distributed computing methods

Paper	Method	Workload	Result	Limitation
AlHafez & Kurdi (2025) [19]	MPI, OpenMP, CUDA survey	General HPC domains	Qualitative analysis only	No IR workload tested
Kang et al. (2015) [20]	OpenMP, MPI, MapReduce	Graph, DB join	Cluster timing comparison	Not text retrieval
Guana & Davidson (2012) [21]	MPI vs. Hadoop	Inverted index	Hadoop faster	No size-aware balancing
Silveira et al. (2018) [22]	CPU+GPU cluster	Inverted index	60× speedup	Needs GPU cluster
NASA (2003) [23]	MPI, OpenMP, Hybrid	NAS benchmarks	Standard baseline	Predates text workloads
Spell-check (2012) [24]	Parallel spell-check	N-Grams corpus	3-stage algorithm	Not in indexing loop
Mei et al. (2015) [26]	Co-occ. data structures	Web IT corpus	Faster lookup	Not parallel
Zhang & Ding (2024) [27]	Apache Spark	Social media text	5–8× faster	Needs big-data tool
Piereck et al. (2020) [28]	HPC text mining	MEDLINE abstracts	2yr reduced to 6 days	Biomedical only
Della Croce et al. (2018) [7]	LPT scheduling theory	Theoretical	Improved bound	Not applied
Li (2018) [31]	MPI/OpenMP vs. Spark	Word count	10× faster	Not IR-specific
OpenMP sched. (2019) [29]	Custom scheduling proposal	None	N/A	Not evaluated
Flowshop (2008) [30]	Static/dynamic balancing	Flowshop scheduling	N/A	Not documents
Manning et al. (2008) [1]	Inverted index theory	Textbook	Defines algorithm	Sequential source
Robertson & Zaragoza (2009) [2]	BM25/TF-IDF	Textbook	Defines formula	Sequential source
Pennington et al. (2014) [3]	Co-occ. counting (GloVe)	Web text	Defines algorithm	Sequential source
Proposed Work	4 models, LPT	49 books	21.5× speedup	—

been proposed in isolation, but no prior study folds it directly into a parallel indexing loop behind a shared, synchronized cache. And while Longest Processing Time scheduling is well established in scheduling theory, it is rarely tested against a concrete, measured information retrieval workload rather than a theoretical bound. This project closes all three gaps at once, inside a single system. The last three rows of Table I, Manning et al. [1], Robertson and Zaragoza [2], and Pennington et al. [3], are not competing parallel systems at all; they are the sequential algorithms this work parallelizes, for inverted-index construction, TF-IDF ranking, and co-occurrence counting respectively. They are listed here to make clear that the contribution of this paper is the parallelization, scheduling, and synchronization layered around established sequential foundations, not the foundations themselves.

III. SEQUENTIAL BASELINE

This section lays out the sequential algorithm that all four parallel versions are built on top of, together with its pseudocode, its complexity, and the baseline runtime measured against it. For a document collection D and dictionary W , the sequential pipeline runs through the following stages:

Algorithm 1 Sequential Information Retrieval Pipeline

- 1: Load dictionary W into a hash table, bucketed by word length
 - 2: Load document collection D
 - 3: **for** each document $d \in D$ **do**
 - 4: **for** each token t extracted from d **do**
 - 5: **if** $t \notin W$ **then**
 - 6: Find the closest word in W within edit distance 2, using cached corrections where available
 - 7: **end if**
 - 8: Insert corrected token into the inverted index for d
 - 9: Update the sliding co-occurrence window for d
 - 10: **end for**
 - 11: **end for**
 - 12: **for** each term t in the inverted index **do**
 - 13: Compute document frequency $df(t)$
 - 14: Compute TF-IDF score for every posting of t
 - 15: **end for**
 - 16: Execute keyword search queries by accumulating TF-IDF scores per document
 - 17: Rank and report top co-occurring word pairs =0
-

Let T be the total number of tokens in the collection, M the number of tokens that need spell correction, L the longest word length, B the number of dictionary words within two characters of a given length, and K the co-occurrence window size. Tokenizing and inserting into the index both run in $O(T)$ time, since hash-table lookups are constant time on average. A spell correction costs $O(B \cdot L^2)$ when it has to run the full edit-distance search, but only $O(1)$ once the result is cached. Updating the co-occurrence window across the whole collection costs $O(T \cdot K)$, and TF-IDF computation costs $O(V)$, where V is the total number of postings across all terms. Putting these together, the overall time complexity is:

$$O(T \cdot K + M \cdot B \cdot L^2 + V) \quad (1)$$

Space is dominated by the inverted index and the co-occurrence table, both built as separate-chaining hash tables, the same structure used in standard treatments of inverted file organization [15]: $O(V + C)$ for the two of them combined, where C is the number of distinct co-occurring word pairs, plus $O(|W|)$ to hold the dictionary

itself. The asymptotic bounds above follow the standard rules for analyzing hash-table and nested-loop costs [12]. Running this sequential version on the test machine described in Section V against the full forty-nine book corpus took 2162.77 seconds.

IV. PARALLEL DESIGN

All three required models, plus the bonus hybrid model, build on the same underlying algorithm and data structures from Section III. What changes between them is how documents get distributed to workers, how the shared spell-correction cache is protected, and how each worker's partial results get merged back into one global index. The dataset itself, including the size differences between individual documents that motivate the load-balancing approach described below, is detailed in Section V.

A. Pthreads Implementation

Documents are split at the level of individual files. Instead of handing each thread a fixed, pre-decided range of documents, every thread repeatedly grabs the next unprocessed document index off a shared counter guarded by a mutex. This is effectively a simple work-stealing scheme, and it balances load across unevenly sized documents without needing to know document sizes in advance.

This implementation has two points where threads have to coordinate. The shared spell-correction cache is guarded by one mutex, and lookups hold that lock only long enough to check whether an entry already exists; on a miss, the lock is released before the costlier edit-distance search runs, then reacquired briefly to store the result. The merge phase works the same way: the global hash table is split into buckets, and threads claim bucket indices through the identical mutex-protected counter pattern used for documents. Because each bucket is owned by exactly one thread at a time, no lock is needed while a thread is actually writing into its bucket. Once the merge finishes, TF-IDF scoring runs as its own parallel pass: the term table is split into fixed ranges and a separate pool of threads computes the document frequency and TF-IDF weight for every posting in its assigned range, so scoring never falls back to a single thread.

B. OpenMP Implementation

OpenMP expresses the same document-level split, but through a parallel for loop with dynamic scheduling instead of hand-written thread logic, letting the OpenMP runtime hand out documents to threads as they free up rather than in fixed blocks. The spell-correction cache is guarded with a named critical section in place of an explicit mutex object. Merging is again a parallel

Algorithm 2 Pthreads Document-Claiming Worker

```
1: shared next_doc  $\leftarrow$  0, protected by mutex  $m$ 
2: function WORKER(thread_id)
3: while true do
4:   LOCK( $m$ );  $d \leftarrow$  next_doc; next_doc  $\leftarrow$  next_doc + 1; UNLOCK( $m$ )
5:   if  $d \geq$  num_documents then
6:     break
7:   end if
8:   PROCESSDOCUMENT( $d$ , local_tables[thread_id])
9: end while
```

for loop, this time over hash-table bucket indices with dynamic scheduling, and the same one-thread-per-bucket rule from the Pthreads version means no extra locking is needed inside the loop. TF-IDF scoring is a third parallel for loop over the same bucket indices, with each thread computing document frequency and TF-IDF weights for the postings in its share of buckets, so this stage stays parallel rather than collapsing onto one thread after the merge.

Algorithm 3 OpenMP Parallel Document Processing

```
1: #pragma omp parallel for
   schedule(dynamic)
2: for  $d = 0$  to num_documents - 1 do
3:   PROCESSDOCUMENT( $d$ , local_tables[omp_get_thread_num()])
4: end for
```

C. MPI Implementation

MPI processes do not share memory, so document assignment has to be settled before execution starts rather than claimed on the fly. Simply handing out documents round-robin breaks down once document sizes vary a lot, since one process can end up stuck with a disproportionately large document while the others finish and sit idle. The fix is to sort documents by size, largest first, a step whose cost is governed by the same comparison-sort bounds covered in standard algorithm references [13], and assign each one greedily to whichever process currently holds the smallest accumulated workload, which is the Longest Processing Time heuristic [6].

Each process builds its own private index and co-occurrence table for its assigned documents, with zero inter-process communication during the build phase. Once that finishes, token counts and timing statistics get combined with MPI_Reduce. Each process then serializes its local index and co-occurrence table into a flat text buffer and sends it to the root process with MPI_Gatherv, chosen specifically because it handles

Algorithm 4 Longest Processing Time Document Assignment

```
1: Sort documents by size in descending order
2: Initialize load[ $p$ ]  $\leftarrow$  0 for all processes  $p$ 
3: for each document  $d$  in sorted order do
4:    $p^* \leftarrow \arg \min_p \text{load}[p]$ 
5:   Assign  $d$  to process  $p^*$ 
6:   load[ $p^*$ ]  $\leftarrow$  load[ $p^*$ ] + size( $d$ )
7: end for
```

the variable-length buffers that different processes produce. From this point on, everything runs on a single process: the root deserializes and merges the gathered buffers, computes TF-IDF scores for every posting with a single sequential pass over the term table, and then runs the keyword search queries. Unlike the Pthreads and OpenMP versions, neither the merge nor the scoring step is split across multiple workers once the data reaches the root.

D. Hybrid MPI and OpenMP Implementation

The bonus implementation stacks both models on top of each other. LPT scheduling distributes documents across MPI processes exactly as above, and then OpenMP dynamic scheduling further splits each process's share of documents across its own threads. Merging happens in two passes: thread-local tables inside each process are merged first, using the same lock-free bucket-partitioning approach described earlier, and then the resulting per-process tables are gathered and merged at the root exactly as in the pure MPI version. TF-IDF scoring, however, is not split across the second OpenMP layer at all: the root process computes it with the same sequential, single-threaded pass used by the pure MPI version, even though that process still has its OpenMP threads available at that point. This is a design gap rather than a hardware limit, and it is one of the reasons the hybrid configurations do not close the gap with Pthreads or OpenMP in Section VI-A.

E. Methodology Overview

Figure 1 lays out the full pipeline end to end: loading the documents and dictionary, distributing documents either through dynamic claiming or Longest Processing Time scheduling depending on the model, running the per-document tokenize, spell-correct, index, and co-occurrence steps inside each worker, merging everything back together, scoring with TF-IDF, and finally producing keyword search and co-occurrence output. The merge and scoring stages are not identical across models: Pthreads and OpenMP keep both stages parallel, splitting the term table into ranges and assigning them across threads, while MPI and Hybrid gather all data to a single

root process and run both the merge and the TF-IDF computation there as a sequential, single-threaded step.

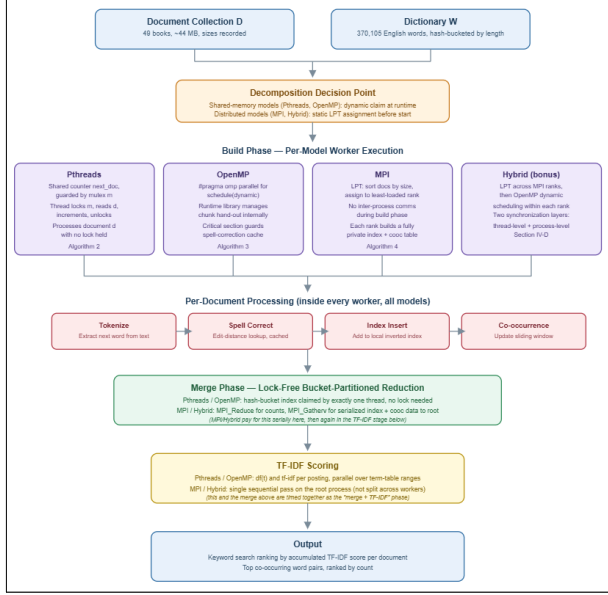


Fig. 1: Methodology pipeline showing document distribution, parallel worker execution under each of the four models, the merge and TF-IDF stages (parallel for Pthreads/OpenMP, sequential on the root process for MPI/Hybrid), and output generation.

V. EXPERIMENTAL SETUP

A. Dataset

The test corpus is forty-nine public-domain books pulled from Project Gutenberg, about forty-four megabytes of natural-language text in total, run against an English dictionary of 370,105 words. Document sizes vary a great deal across the corpus, with the largest document more than twenty times the size of the smallest. That size spread was kept deliberately, since it is exactly what motivates the load-balancing piece of the parallel design in Section IV. This corpus size was chosen to fit the hardware and time budget available for this project rather than to represent the upper limit of what any of the four implementations can handle; the speedup and efficiency numbers in Section VI should be read as a comparison between paradigms on a fixed, moderately sized corpus, not as a claim about absolute throughput at production scale.

B. Hardware and Software

C. Input Sizes Tested

For strong scaling, the input is kept fixed at the full forty-nine book corpus (roughly forty-four megabytes of text) while the worker count is varied across one, two, four, and eight workers for each model. For weak

TABLE II: Hardware and Software Configuration

Parameter	Value
CPU	Intel Core i5-8350U @ 1.70GHz
Cores / Threads	4 cores, 8 threads (hyperthreaded)
L1d cache	128 KiB (4 instances)
L2 cache	1 MiB (4 instances)
L3 cache	6 MiB (shared)
RAM	7.7 GiB
Operating System	Ubuntu 24.04.1 LTS (WSL2)
Compiler	GCC 13.3.0, flags <code>-O2 -Wall</code>
MPI Library	Open MPI 4.1.6

scaling, the input grows along with the worker count instead: six books at one worker, twelve at two, twenty-four at four, and forty-eight at eight, so the amount of work per worker stays roughly constant if scaling were perfect. Every configuration was run three times, and the values reported are the average of those three runs.

VI. RESULTS

A. Strong Scaling

Table III lists the mean execution time, standard deviation, and speedup over the sequential baseline for every configuration tested.

TABLE III: Strong Scaling Results (49-book corpus)

Configuration	Workers	Mean (s)	Std (s)	Speedup
Sequential	1	2162.77	0.00	1.00×
Pthreads	1	614.60	34.69	3.52×
Pthreads	2	270.51	10.43	8.00×
Pthreads	4	129.39	26.25	16.71×
Pthreads	8	100.56	10.52	21.50×
OpenMP	2	270.24	33.07	8.00×
OpenMP	4	158.81	6.66	13.62×
OpenMP	8	133.10	10.26	16.25×
MPI	2	312.73	38.17	6.92×
MPI	4	188.84	10.97	11.45×
MPI	8	191.17	4.67	11.31×
Hybrid (2p×4t)	8	179.64	11.63	12.04×
Hybrid (4p×2t)	8	144.14	24.08	15.00×
Hybrid (8p×1t)	8	182.67	6.40	11.84×

Pthreads comes out on top of all four models at eight workers, hitting 21.50 times the sequential baseline and beating OpenMP's 16.25 times at the same worker count. The most likely reason is granularity: each Pthreads worker checks out exactly one document index at a time through an explicit mutex with no batching overhead, while OpenMP's dynamic scheduling clause carries extra bookkeeping inside its compiler-generated runtime to manage chunk distribution. At lower worker counts the two are close to each other, since either approach's overhead is small next to the dominant cost of the build phase itself. MPI trails both shared-memory models at every worker count, which comes down to the extra serialization and communication cost in its gather-based merge step.

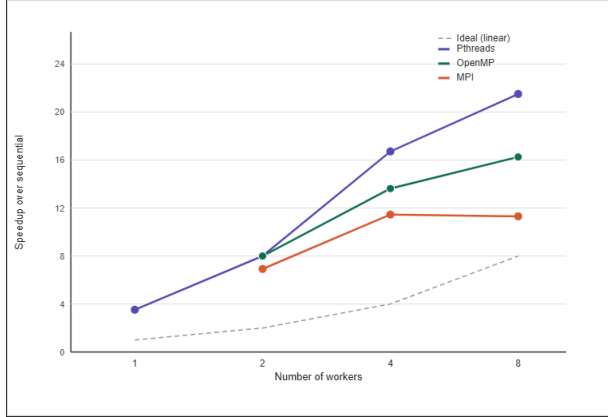


Fig. 2: Strong scaling speedup versus number of workers for Pthreads, OpenMP, and MPI, against the ideal linear speedup line.

Figure 2 plots the same numbers. All three curves sit below the ideal line and drift further from it as the worker count climbs, which lines up with the Amdahl’s Law argument in Section VII-A. Pthreads tracks closest to ideal the whole way, OpenMP pulls away more noticeably between four and eight workers, and MPI nearly flattens out over that same range, which is the clearest visual sign that MPI gets the least benefit out of the second doubling of workers.

B. Time Breakdown

The execution time of each parallel version is composed of a build phase, during which threads or processes independently index their assigned documents, and a combined merge-and-scoring phase, during which partial results are combined and TF-IDF scores are computed for every posting. These two stages are timed together in the implementation, so the numbers below cover both. On the 49-book corpus at eight workers, Pthreads’ merge-and-scoring phase averages 2.25 seconds across the three repeated runs, and OpenMP’s averages 2.11 seconds, both consistent with the fact that the bucket-partitioned merge and the TF-IDF pass that follows it stay split across all eight threads in both implementations. MPI’s merge-and-scoring phase at eight processes averages 12.57 seconds, roughly six times longer, because for MPI neither stage is split across workers: the gather-based merge and the TF-IDF computation both run as a single sequential pass on the root process while the remaining seven processes sit idle. The equivalent breakdown for Hybrid was not separately logged in this set of experiments, so it is not reported here.

C. Weak Scaling

For the weak scaling experiment, dataset size scales up proportionally with worker count: six books per worker

at one worker, all the way up to forty-eight books at eight workers, so the work per worker would stay roughly constant under perfect scaling. Table IV shows what actually happened.

TABLE IV: Weak Scaling Results (dataset size scales with workers)

Workers	Books	Total Time (s)	Efficiency
<i>Pthreads</i>			
1	6	17.68	1.000
2	12	20.93	0.844
4	24	41.04	0.431
8	48	79.76	0.222
<i>OpenMP</i>			
1	6	21.31	1.000
2	12	22.42	0.950
4	24	41.16	0.518
8	48	94.25	0.226
<i>MPI</i>			
1	6	23.94	1.000
2	12	29.78	0.804
4	24	40.46	0.592
8	48	168.30	0.142

Weak scaling efficiency here is just the one-worker time divided by the time at a given worker count, so a value of 1.0 means perfectly flat scaling as workers and data double together.

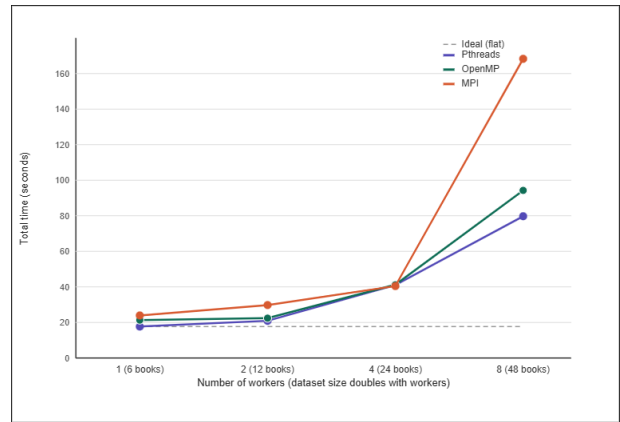


Fig. 3: Total execution time versus number of workers under weak scaling, where dataset size doubles alongside worker count. The dashed line marks ideal (flat) weak scaling.

Figure 3 shows none of the three models hold flat under weak scaling, and all three drop off most sharply in the last doubling, from four to eight workers. MPI takes the worst hit by far, falling to an efficiency of 0.142 at eight workers and more than quadrupling in time between four and eight workers, from 40.46 to 168.30 seconds. Pthreads and OpenMP fall off more gently and land at nearly the same efficiency, 0.222 and 0.226, even though Pthreads was the clear winner under strong scaling. Taken together, this suggests the

per-worker build cost scales acceptably for all three, but MPI’s gather-based merge becomes disproportionately expensive as the number of processes and the volume of serialized index data both grow at once, since every process still has to funnel its data through a single root no matter how many are sending it.

VII. DISCUSSION

A. Why Speedup Plateaus

Every model shows diminishing returns past four workers, which fits Amdahl’s Law [5], but the size of the serial fraction is not the same across models. For Pthreads and OpenMP, both the merge and the TF-IDF scoring pass stay split across every thread, so the only serial work left is the small bookkeeping between phases; their plateau comes mainly from build-phase work shrinking faster than the fixed per-thread overhead as worker count grows. For MPI and Hybrid, the serial fraction is genuinely larger, because two separate stages, the gather-based merge and the TF-IDF computation, both run as a single sequential pass on the root process while every other process sits idle. MPI also carries communication overhead the shared-memory models don’t have at all: serializing, sending, and deserializing index data through `MPI_Gatherv` costs more as both the index size and the process count grow.

B. Pthreads Outperforming OpenMP

One result worth dwelling on is that Pthreads beats OpenMP’s speedup at eight workers, 21.50 times against 16.25 times, even though both use the same document-claiming idea and the same bucket-partitioned merge. Scheduling overhead is the likely explanation: OpenMP’s `schedule(dynamic)` clause runs through the compiler’s runtime library, which does its own internal bookkeeping to hand out chunks across threads, while the Pthreads version uses one explicit mutex around a plain integer counter with no layer in between. With only forty-nine units of work spread across eight threads, that difference in overhead is proportionally large enough to show up in the numbers. The takeaway is that for fine-grained, document-level parallelism over a small number of sizeable work units, hand-written synchronization can beat compiler-managed scheduling, which runs against the usual assumption that OpenMP and Pthreads should perform about the same.

C. Load Imbalance and the Effect of LPT Scheduling

The test corpus was deliberately kept uneven in document size, with the largest document more than twenty times the size of the smallest. Under plain round-robin assignment, whichever process or thread lands the oversized document keeps working long after everyone else has finished, leaving cores idle in the meantime. The

LPT scheduler at the MPI level fixes this by handing out the largest documents first, always to whichever process is currently carrying the least load, a strategy Graham proved keeps the resulting makespan within four-thirds of the theoretical optimum minus a term that shrinks as worker count grows [6]. OpenMP gets a similar effect for free, since idle threads simply grab the next available document without needing any size-aware ordering at all.

D. Programming Effort Comparison

Among the three required models, OpenMP demands the least change from the sequential baseline, since its parallel regions are just compiler directives sitting on top of otherwise sequential code. Pthreads asks for explicit thread management, mutex handling, and work-distribution logic that OpenMP’s runtime otherwise takes care of automatically, which makes for a longer and more error-prone implementation to get the same functionality. That extra effort is not wasted, though: it is the direct reason for the performance edge seen in Section VI-A, since the same fine-grained control that makes Pthreads harder to write is also what removes the scheduling overhead holding OpenMP back at high thread counts. MPI demands the biggest rewrite of the three, since having no shared memory forces explicit serialization, communication, and merge logic that simply has no counterpart in either of the other models. The hybrid version piles on top of that again, since it asks the programmer to juggle two separate levels of parallelism and two separate synchronization mechanisms at the same time.

VIII. CONCLUSION

This paper parallelized a full information retrieval pipeline, inverted indexing, spell correction, TF-IDF scoring, keyword search, and co-occurrence analysis, across Pthreads, OpenMP, and MPI, plus a bonus hybrid version for further comparison. A Longest Processing Time scheduler at the MPI level corrected the load imbalance caused by uneven document sizes, and a lock-free bucket-partitioned merge kept synchronization overhead down during result aggregation in every model. On the forty-nine book test corpus, Pthreads came out ahead of all four models, reaching roughly 21.50 times the sequential baseline at eight threads, ahead of OpenMP’s roughly 16.25 times and MPI’s roughly 11.45 times at four processes. That result says something worth remembering: hand-written synchronization primitives can beat compiler-managed dynamic scheduling for fine-grained, document-level parallelism over a small number of sizeable work units. The main limits turned out to be the serial fraction of the merge and scoring stages, in line with Amdahl’s Law, and the communication overhead that comes specifically with the distributed-memory MPI

model. One direction worth pursuing next is extending the lock-free bucket-partitioned merge to the MPI gather phase itself, perhaps through a tree-based or distributed reduction instead of a single-root gather, which could narrow the gap between MPI and the shared-memory models at higher process counts.

REFERENCES

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.
- [2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [3] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. EMNLP*, 2014, pp. 1532–1543.
- [4] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Phys. Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [5] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
- [6] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM J. Appl. Math.*, vol. 17, no. 2, pp. 416–429, 1969.
- [7] F. Della Croce and R. Scatamacchia, "Longest processing time rule for identical parallel machines revisited," *arXiv preprint arXiv:1801.05489*, 2018.
- [8] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 2nd ed. Cambridge, MA: MIT Press, 1999.
- [9] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA: MIT Press, 2007.
- [10] D. R. Butenhof, *Programming with POSIX Threads*. Boston, MA: Addison-Wesley, 1997.
- [11] P. S. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA: Morgan Kaufmann, 2011.
- [12] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [13] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Boston, MA: Addison-Wesley, 1998.
- [14] J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Comput. Surv.*, vol. 38, no. 2, p. 6, 2006.
- [15] I. H. Witten, A. Moffat, and T. C. Bell, *Managing Gigabytes: Compressing and Indexing Documents and Images*, 2nd ed. San Francisco, CA: Morgan Kaufmann, 1999.
- [16] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. OSDI*, 2004, pp. 137–150.
- [17] M. J. Flynn, "Some computer organizations and their effectiveness," *IEEE Trans. Comput.*, vol. C-21, no. 9, pp. 948–960, 1972.
- [18] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Tech. Rep. 1999-66, 1999.
- [19] N. AlHafez and A. Kurdi, "Parallel paradigms in modern HPC: A comparative analysis of MPI, OpenMP, and CUDA," *arXiv preprint arXiv:2506.15454*, 2025.
- [20] S. J. Kang, S. Y. Lee, and K. M. Lee, "Performance comparison of OpenMP, MPI, and MapReduce in practical problems," *Adv. Multimedia*, vol. 2015, Art. no. 575687, 2015.
- [21] V. Guana and J. Davidson, "On comparing inverted index parallel implementations using Map/Reduce," Univ. of Alberta, Tech. Rep., 2012.
- [22] T. B. Silveira, F. A. L. Soares, W. C. Brandão, and H. C. Freitas, "Heterogeneous parallel architecture for inverted index generation," in *Proc. Conf. Comput. Sci.*, PUC Minas, 2018.
- [23] H. Jin, M. Frumkin, and J. Yan, "Comparing the OpenMP, MPI, and hybrid programming paradigms on an SMP cluster," NASA Advanced Supercomputing Division, Tech. Rep. NAS-03-019, 2003.
- [24] "Parallel spell-checking algorithm based on Yahoo! N-grams dataset," *arXiv preprint arXiv:1204.0184*, 2012.
- [25] "Accelerating spell checking with parallel processing techniques," in *Proc. IEEE Conf.*, 2025.
- [26] J. Mei, X. Kou, Z. Yao, A. Rau-Chaplin, A. Islam, A. Moh'd, and E. E. Milios, "Efficient computation of co-occurrence based word relatedness," in *Proc. ACM DocEng*, 2015, pp. 43–46.
- [27] X. Zhang and T. Ding, "Co-occurrence word model for news media hotspot mining: Text mining method design," *Math. Biosci. Eng.*, vol. 21, no. 4, pp. 5411–5429, 2024.
- [28] B. Pioreck, M. Oliveira-Lima, A. M. Benko-Iseppon, S. Diehl, R. Schneider, A. C. Brasileiro-Vidal, and A. Barbosa-Silva, "LAITOR4HPC: A text mining pipeline based on HPC for building interaction networks," *BMC Bioinformatics*, vol. 21, p. 365, 2020.
- [29] "Toward a standard interface for user-defined scheduling in OpenMP," *arXiv preprint arXiv:1906.08911*, 2019.
- [30] "Load balancing strategies to solve flowshop scheduling on parallel computing," *arXiv preprint arXiv:0809.3285*, 2008.
- [31] J. Li, "Comparing Spark vs MPI/OpenMP on word count MapReduce," *arXiv preprint arXiv:1811.04875*, 2018.
- [32] R. Rabenseifner, G. Hager, and G. Jost, "Hybrid MPI/OpenMP parallel programming on clusters of multi-core SMP nodes," in *Proc. 17th Euromicro Int. Conf. Parallel, Distrib. Netw.-Based Process.*, 2009, pp. 427–436.
- [33] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.